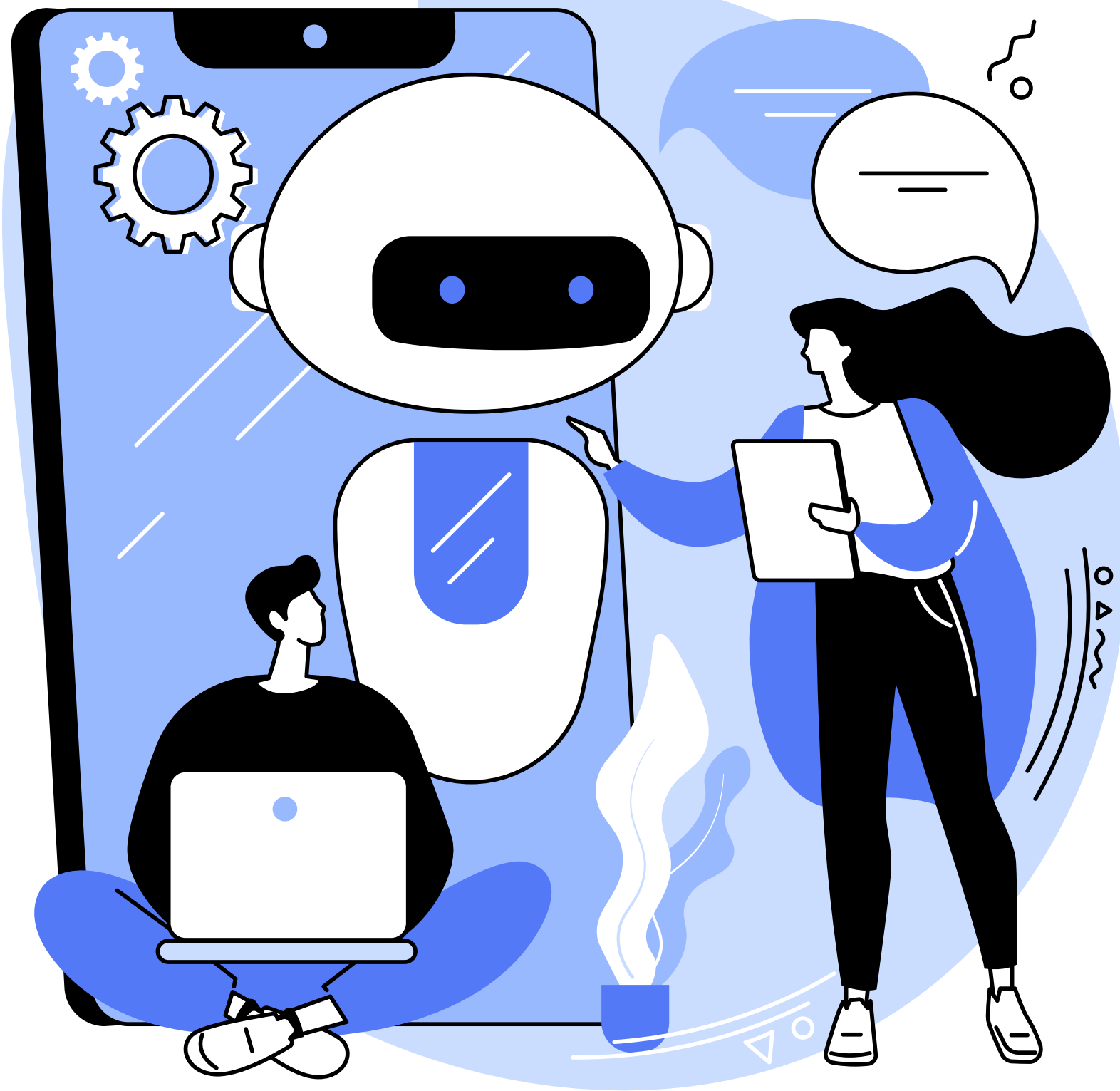




MOBILENET IMPLEMENTATION

```
(base) enescaglar@Enes-MacBook-Air mobileNetImplementation %  
Number of training classes: 200  
Number of images in class n01443537: 500  
Total training images: 100000  
Validation folder structure fixed.  
Total validation images: 10000
```



Tiny ImageNet

Kaggle is the world's largest data science community with powerful tools and resources to help you achieve your data science goals.

[kaggle.com](https://www.kaggle.com)

- create
- home
- competitions
- datasets
- models
- code
- discussions
- learn
- more
- your work
- reviewed
- Movie Dataset: Budge...
- the Movies Dataset

Tiny ImageNet

▲

206

<> Code

Download

⋮

- Data Card
- Code (82)
- Discussion (1)
- Suggestions (0)

tiny-imagenet-200 (3 directories, 2 files)



About this directory

Suggest Edits

This file does not have a description yet.

test
1 directories

train
200 directories

val
1 directories

wnids.txt
2 kB

words.txt

Data Explorer

Version 1 (239.65 MB)

- tiny-imagenet-200
 - test
 - train
 - val
 - wnids.txt
 - words.txt

Summary

- 120k files

🧠 Example class labels used in training:
.DS_Store: Unknown
n01443537: goldfish, *Carassius auratus*

Sample Train Image - Class: n01443537



WordNet ID

**Common English
label**

**Scientific
(Latin) name**

Figure 1

poncho



water jug



bannister



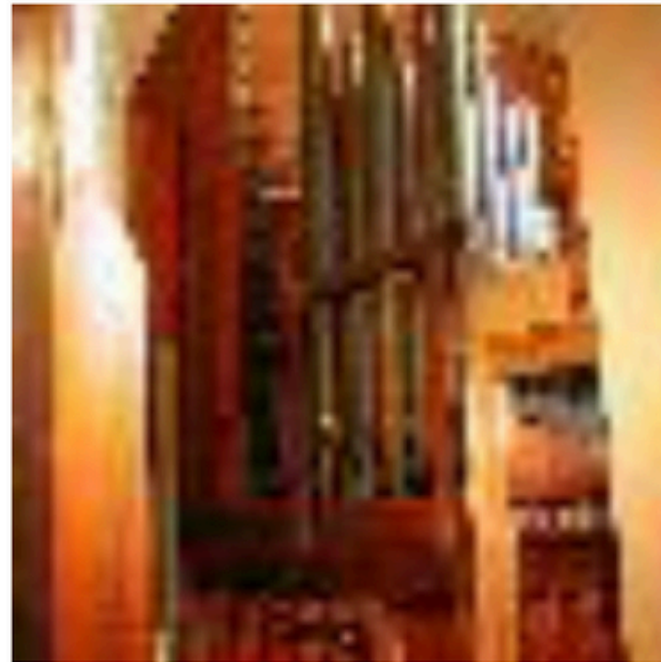
dugong



German shepherd



organ



European fire salamander



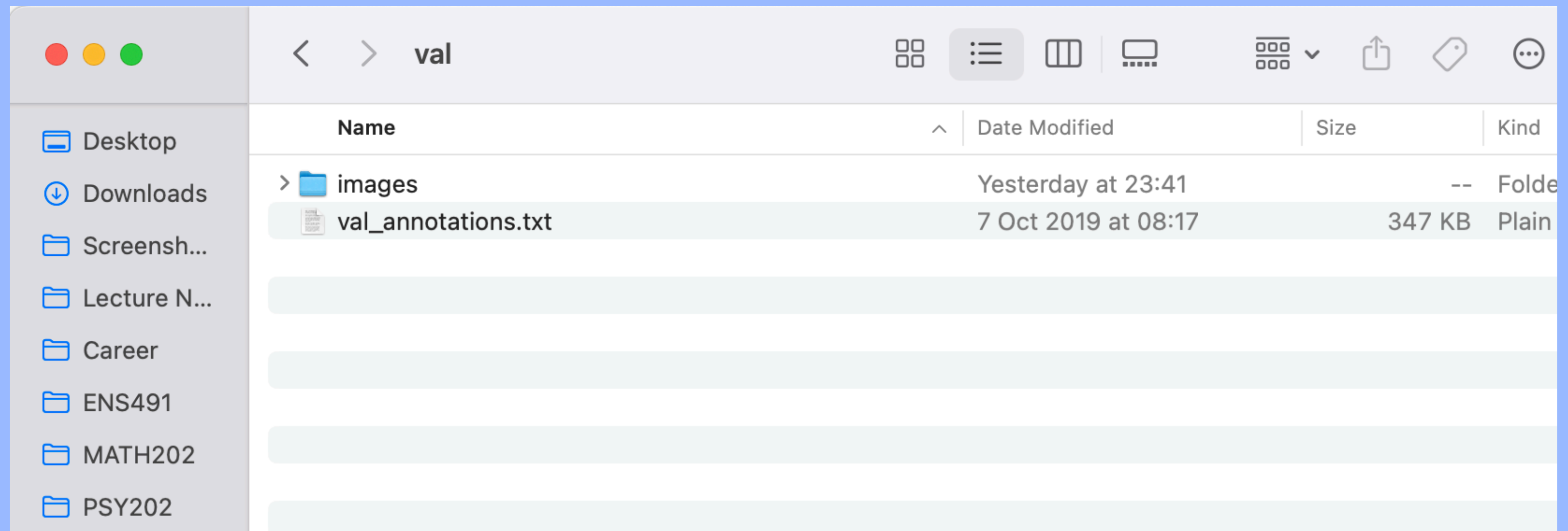
sulphur butterfly



Fixing the Validation Set Structure

Tiny ImageNet's validation images are all stored in one folder with their labels listed separately in `val_annotations.txt`.

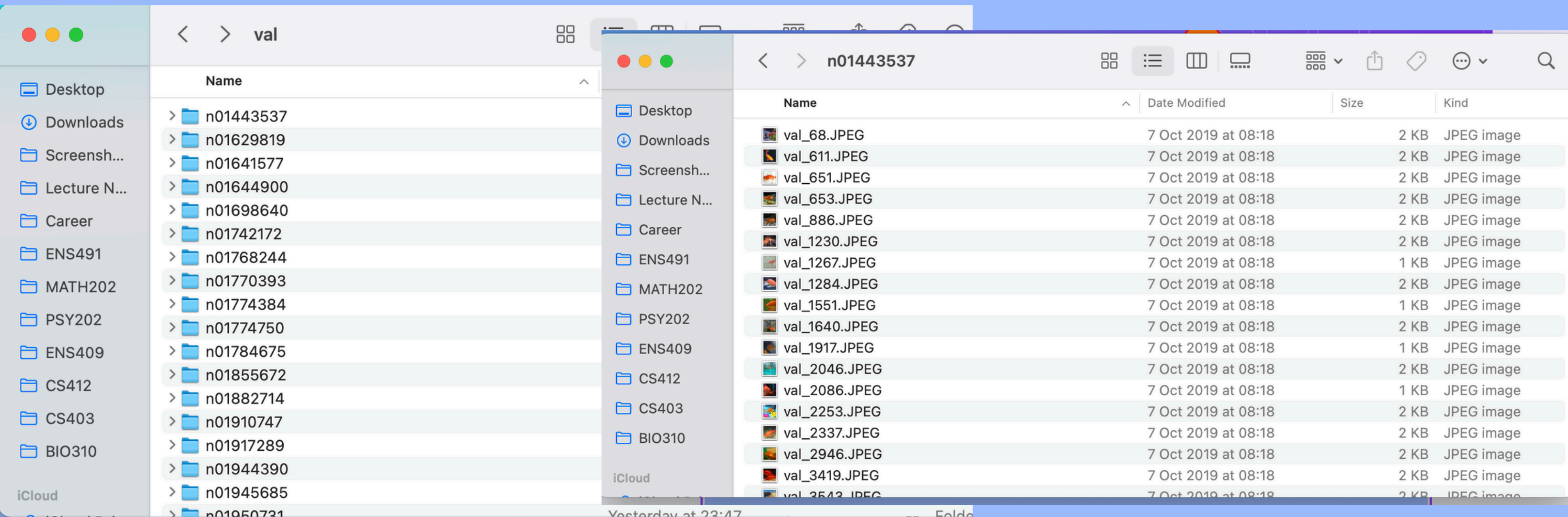
This format is not compatible with PyTorch's ImageFolder loader.



We parsed val_annotations.txt

Moved each image into its corresponding class folder (e.g.,
val/n01443537/)

Deleted the original val/images/ folder



```

torch.set_num_threads(1)

class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride, padding, groups=1, bias=False):
        super().__init__()
        self.c = nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding, groups=groups, bias=bias)
        self.bn = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.relu(self.bn(self.c(x)))

```

in_channels: Number of input channels (e.g., 3 for RGB images)

out_channels: Number of filters to apply

kernel_size: Size of the filter (e.g., 3 for 3x3)

stride: Step size (controls downsampling)

padding: Adds border pixels so output size can be preserved

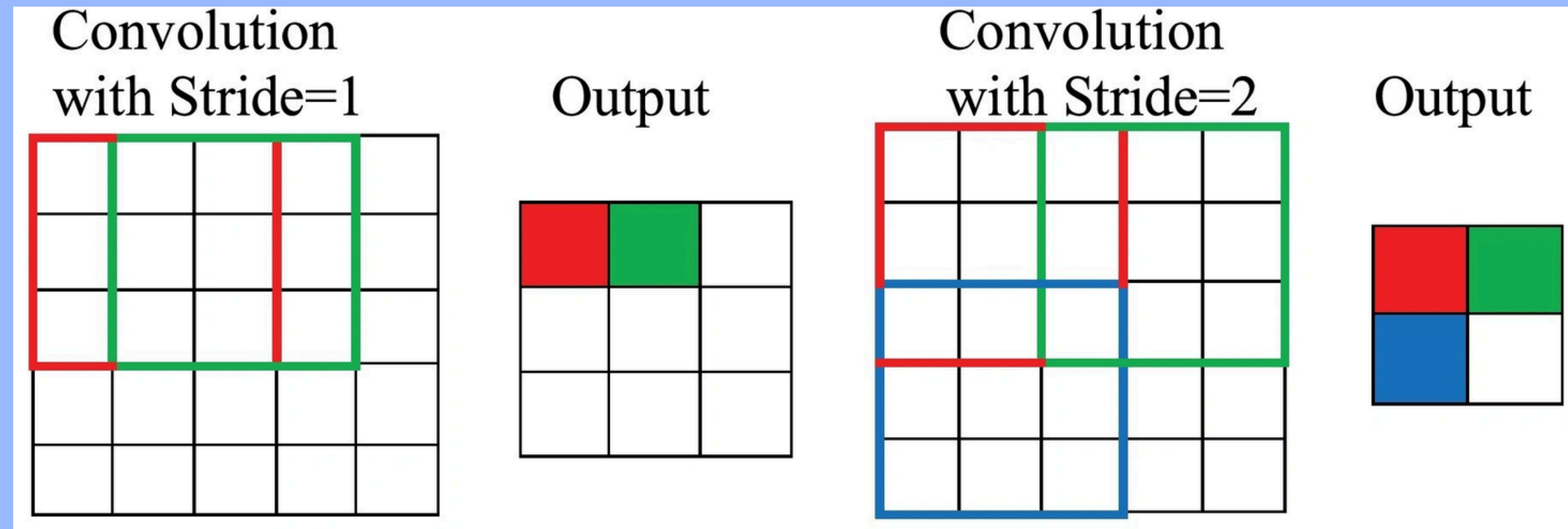
groups: Used for depthwise separable convolutions

If groups=1 → regular convolution

If groups=in_channels → depthwise convolution (each input channel processed independently)

bias=False: Disables the bias term, usually okay since BatchNorm handles offset.

Stride



downsample, reduce spatial size.

Padding

adding border pixels (usually 0s) around the input image.

**No Padding
(padding = 0)
Output
becomes
smaller than
input
You lose
border
information**

padding = 1

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114	328	-26	470	158
53				

Preserve edges

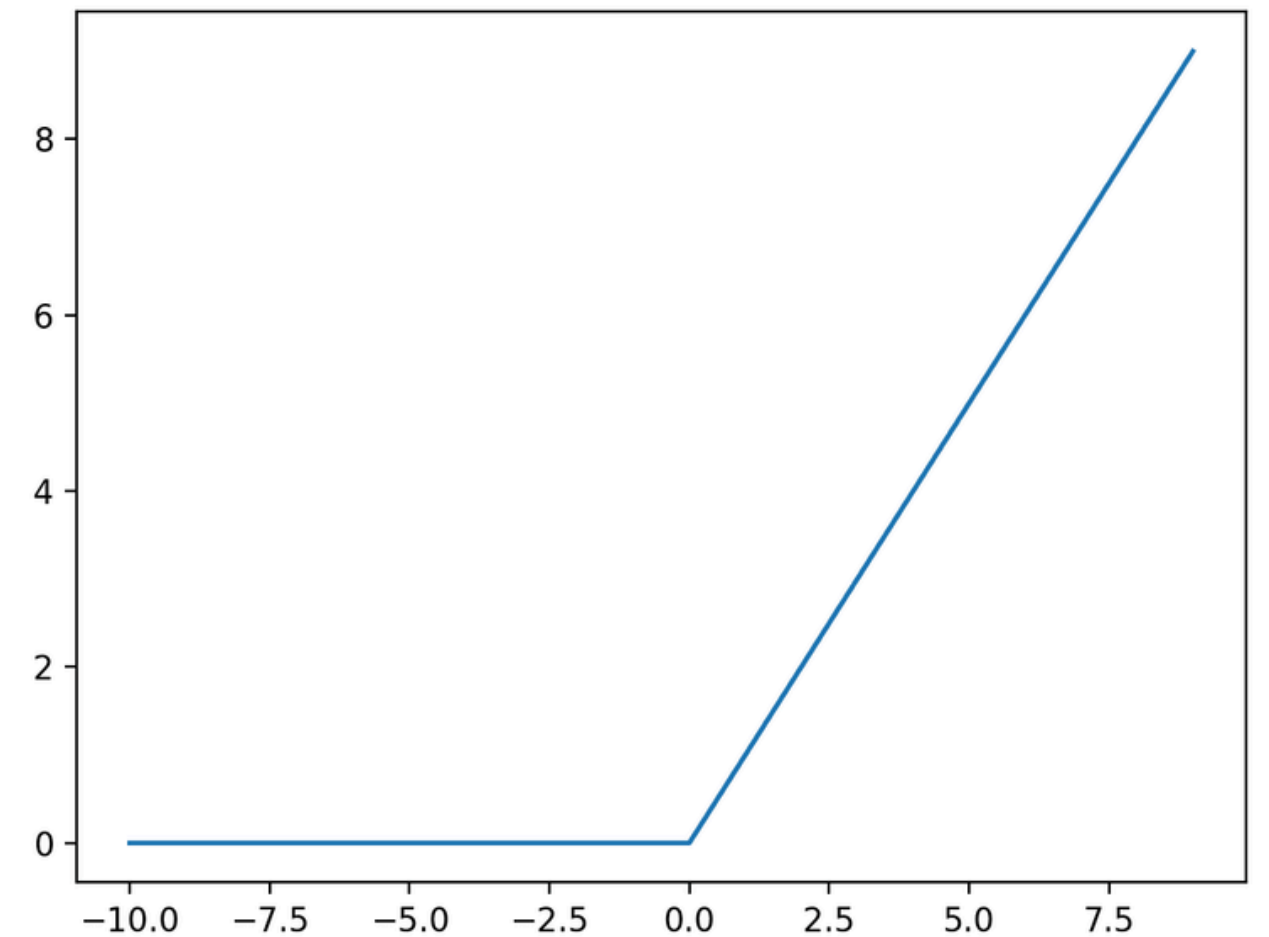
Maintains size

Batch Normalization

To stabilize training by normalizing each batch's output (mean 0, std 1).

ReLU activation function

which introduces non-linearity.



```

class DepthwiseConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride):
        super().__init__()
        self.depthwise = ConvBlock(in_channels, in_channels, 3, stride, 1, groups=in_channels)
        self.pointwise = ConvBlock(in_channels, out_channels, 1, 1, 0)

    def forward(self, x):
        x = self.depthwise(x)
        x = self.pointwise(x)
        return x

```

Input channels = Output channels

Kernel size = 3x3

Stride = passed from the constructor

Padding = 1 (to preserve dimensions if stride=1)

groups=in_channels → this is the key trick: each input channel is convolved separately.

Kernel size = 1x1 → so it's not looking at surrounding pixels

Input channels = all depthwise outputs

Output channels = what you pass in

Stride = 1 (no downsampling)

Padding = 0


```
class MobileNet(nn.Module):
    def __init__(self, alpha=1.0, in_channels=3, classes=200):
        super().__init__()
        strides = [1, 2, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 1]
        channels = [32, 64, 128, 128, 256, 256, 512, 512, 512, 512, 512, 512, 1024, 1024]

        if alpha < 1:
            channels = [int(c * alpha) for c in channels]
```

alpha: Width multiplier that reduces the number of channels (e.g., 0.5 = half channels = faster).

in_channels: Number of input channels (usually 3 for RGB images).

classes: Number of output classes (Tiny ImageNet has 200).

A stride of 2 downsamples the image.

A stride of 1 keeps the spatial size the same.

If alpha is less than 1, reduce each channel by a factor of alpha

```

self.conv1 = ConvBlock(in_channels, channels[0], 3, 2, 1)
self.blocks = nn.Sequential(
    *[DepthwiseConvBlock(channels[i], channels[i + 1], strides[i]) for i in range(len(strides))]
)
self.classifier = nn.Sequential(
    nn.AdaptiveAvgPool2d((1, 1)),
    nn.Flatten(),
    nn.Linear(channels[-1], classes)
)

```

It loops through each pair of channels and stride to define all 13 blocks

```

def forward(self, x):
    x = self.conv1(x)
    x = self.blocks(x)
    x = self.classifier(x)
    return x

```

nn.AdaptiveAvgPool2d((1, 1)): Globally pools each feature map into a single number (regardless of input size).

nn.Flatten(): Converts the 3D output into 1D for the linear layer.

nn.Linear(...): Final linear layer to predict the class scores.

Input: RGB image (3 channels)

Output: first value in channels (e.g. 32)

3x3 kernel

Stride 2 (downsamples)

Padding 1 (keeps size proportional)

ConvBlock: Downsamples the image and extracts low-level features.

13 DepthwiseConvBlocks: Efficiently build deeper features with low parameters.

Classifier Head: Pools and classifies into one of 200 TinyImageNet classes.

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 1024$
	Conv dw / s2	$3 \times 3 \times 1024$ dw
	Conv / s1	$1 \times 1 \times 1024 \times 1024$
	Avg Pool / s1	Pool 7×7
	FC / s1	1024×1000
	Softmax / s1	Classifier

```

class MobileNet(nn.Module):
    def __init__(self, alpha=1.0, in_channels=3, classes=200):
        super().__init__()
        strides = [1, 2, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 1]
        channels = [32, 64, 128, 128, 256, 256, 512, 512, 512, 512, 512, 512, 1024, 1024]

        if alpha < 1:
            channels = [int(c * alpha) for c in channels]

        self.conv1 = ConvBlock(in_channels, channels[0], 3, 2, 1)
        self.blocks = nn.Sequential(
            *[DepthwiseConvBlock(channels[i], channels[i + 1], strides[i]) for i in range(12)]
        )
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(channels[-1], classes)
        )

```



```
# ----- 9. TRAINING -----
```

```
transform = transforms.Compose([  
    transforms.Resize((224, 224)),  
    transforms.ToTensor()  
])
```

```
train_dataset = ImageFolder(train_path, transform=transform)
```

```
val_dataset = ImageFolder(val_path, transform=transform)
```

```
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=0)
```

```
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False, num_workers=0)
```

```
device = torch.device("cpu")
```

```
model = MobileNet().to(device)
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

**Resizes every image to 224x224 pixels
— required by MobileNet input**

**This is the loss function used for
classification tasks.**

**It compares the model's
predictions to the true labels and
computes how wrong it was.**

Adam: An optimizer that adjusts the learning rate for each parameter adaptively.

model.parameters(): Tells the optimizer which parts of the model to update.

lr=0.001: Sets the initial learning rate — how fast the model learns.

 The optimizer will use gradients to update model weights during training.

Results

WHAT'S NEXT?

MobileNet v2 — logical continuation of the first model, developed in 2018.

It is 0.3x faster and smaller, +1% more accurate than MobileNet v1.

MobileNet v3 is the next generation of the MobileNet family, which is full of improvements to the MobileNet v2, developed in 2019.

It is 2x faster, 30% smaller, -3% accurate than MobileNet v2.